

CAP 5. CAMADA DE TRANSPORTE

AULA 1: SERVIÇOS DA CAMADA

INE5422 REDES DE COMPUTADORES II
PROF. ROBERTO WILLRICH (INE/UFSC)
ROBERTO.WILLRICH@UFSC.BR
[HTTPS://MOODLE.UFSC.BR](https://moodle.ufsc.br)

Capítulo 3

Camada de Transporte

Nota sobre o uso destes slides ppt:

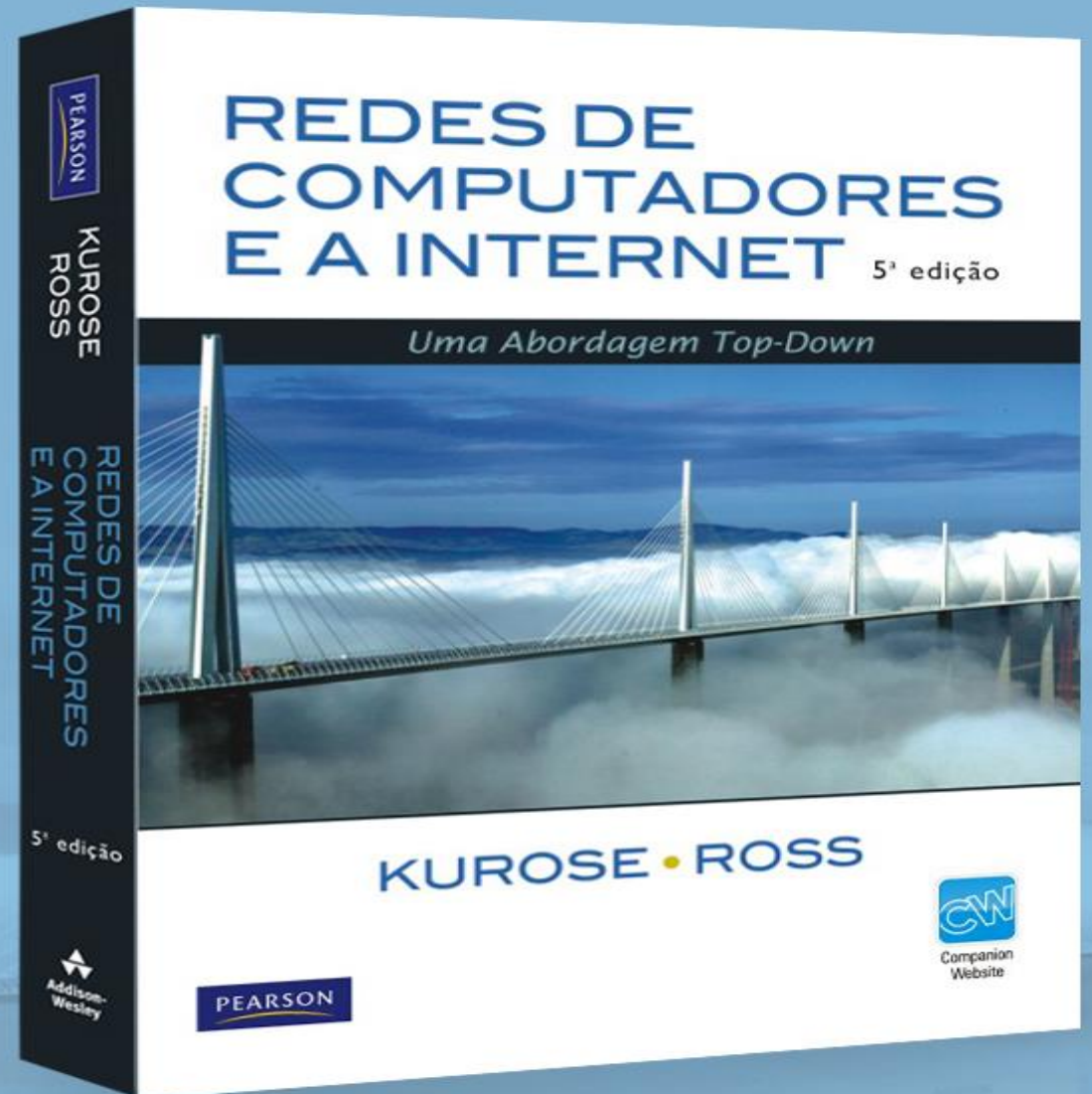
Estamos disponibilizando estes slides gratuitamente a todos (professores, alunos, leitores). Eles estão em formato do PowerPoint para que você possa incluir, modificar e excluir slides (incluindo este) e o conteúdo do slide, de acordo com suas necessidades. Eles obviamente representam *muito* trabalho da nossa parte. Em retorno pelo uso, pedimos apenas o seguinte:

- ❑ Se você usar estes slides (por exemplo, em sala de aula) sem muita alteração, que mencione sua fonte (afinal, gostamos que as pessoas usem nosso livro!).
- ❑ Se você postar quaisquer slides sem muita alteração em um site Web, que informe que eles foram adaptados dos (ou talvez idênticos aos) nossos slides, e inclua nossa nota de direito autoral desse material.

Obrigado e divirta-se! JFK/KWR

Todo o material copyright 1996-2009

J. F Kurose e K. W. Ross, Todos os direitos reservados.



CAPÍTULO 5: CAMADA DE TRANSPORTE

Metas do capítulo:

- Entender os princípios por trás dos serviços da camada de transporte:
 - multiplexação/demultiplexação
 - transferência confiável de dados
 - controle de fluxo
 - controle de congestionamento

Aprender os protocolos de transporte da Internet:

- Transporte sem conexão: UDP
- Transporte orientado a conexão: TCP
 - transferência confiável
 - controle de fluxo e de congestionamento
 - gerenciamento de conexões

SERVIÇOS DA CAMADA DE TRANSPORTE

Finalidade

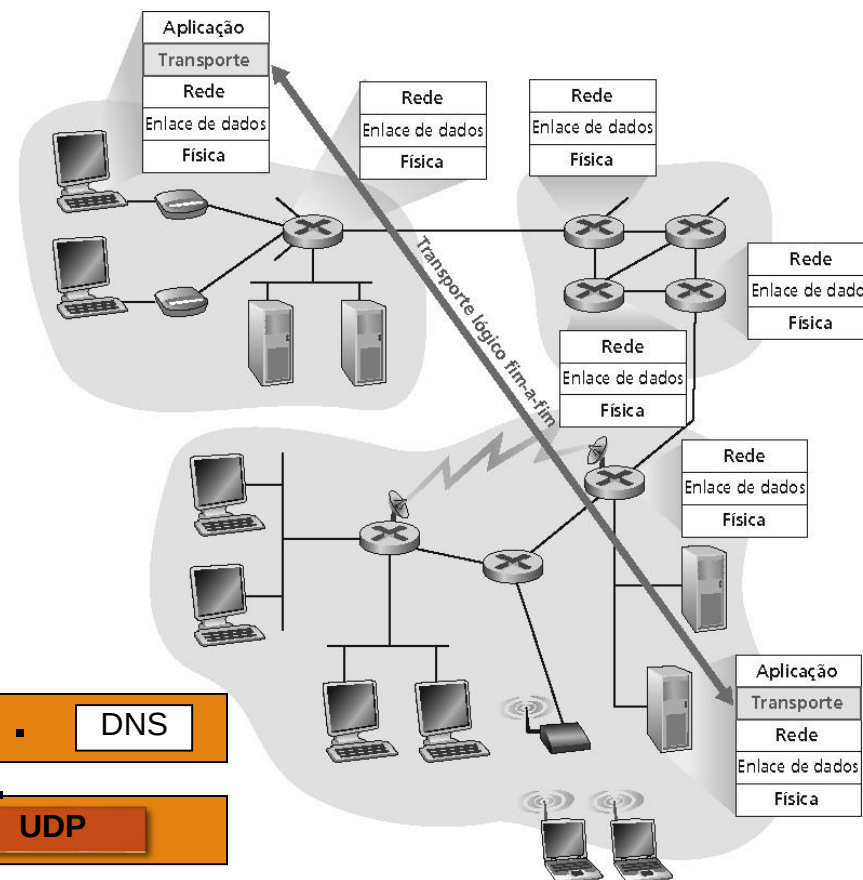
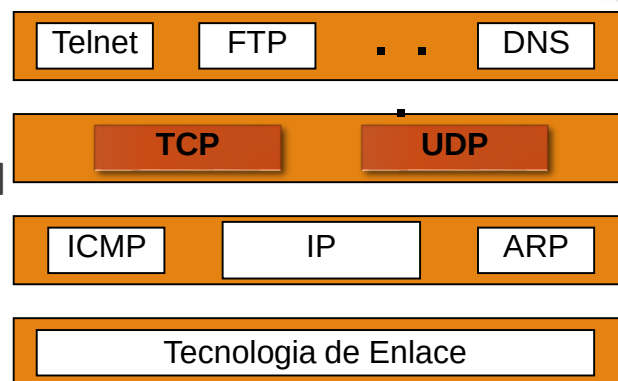
- Fornece comunicação lógica entre processos de aplicação rodando em hosts diferentes

Protocolos de transporte rodam em sistemas finais

- **Lado emissor:** quebra as mensagens da aplicação em segmentos e envia para a camada de rede
- **Lado receptor:** remonta os segmentos em mensagens e passa para a camada de aplicação

Há mais de um protocolo de transporte disponível para as aplicações

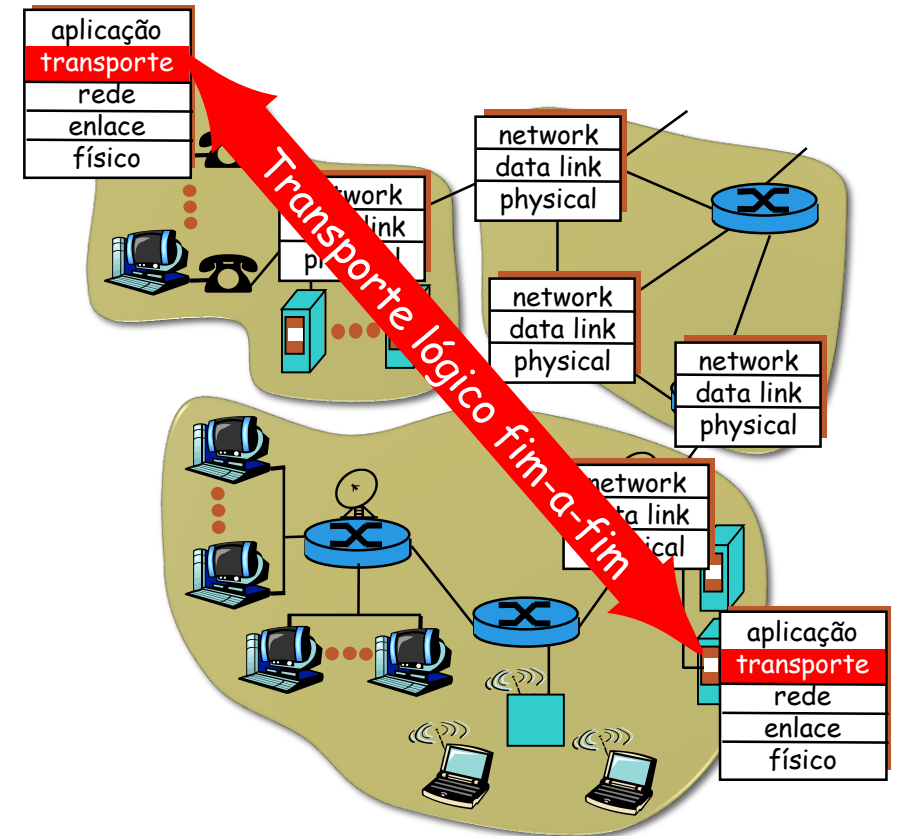
- **UDP – User Datagram Protocol**
 - Serviço sem conexão não confiável
- **TCP – Transmission Control Protocol**
 - Serviço confiável e orientado a conexão



PROTOCOLOS DA CAMADA DE TRANSPORTE

Serviços de transporte da Internet

- **TCP:** Confiável, liberação em ordem unicast
 - Configuração de conexão
 - Sequenciamento de bytes
 - Controle de fluxo e de congestionamento
- **UDP:** Não confiável, liberação fora de ordem unicast ou multicast
- Serviços não disponíveis no TCP e UDP:
 - Tempo-real
 - Garantias de banda
 - Multicast confiável



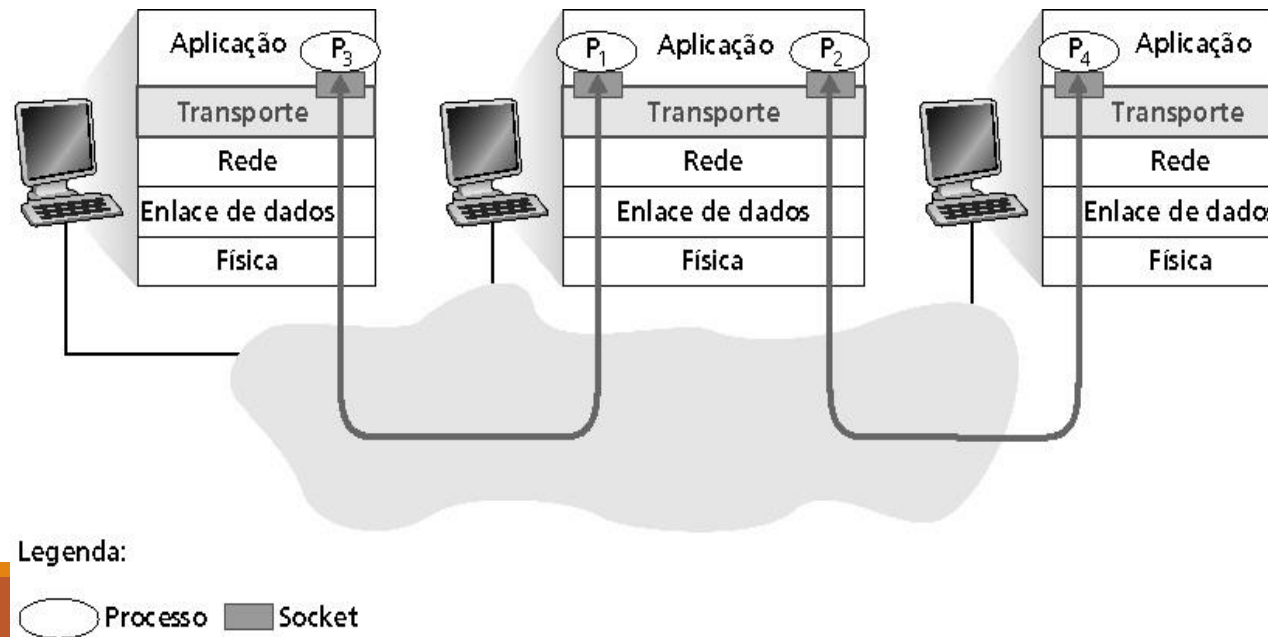
MULTIPLEXAÇÃO/DEMULTIPLEXAÇÃO

Demultiplexação no hospedeiro receptor:

- entrega os segmentos recebidos ao socket correto

Multiplexação no hospedeiro emissor:

- coleta dados de múltiplos sockets, envelope os dados com cabeçalho (usado depois para demultiplexação)



PORTAS DE PROTOCOLO

Último fonte/destino de/para uma mensagem é uma porta de protocolo

- Um processo envia via/ouve uma porta de protocolo (identificado por um inteiro)
- Muitos sistemas operacionais fornecem acesso síncrono as portas
 - Um processo pode se bloquear aguardando chegada de mensagens na porta
- Em geral, portas são buferizadas
 - Dados chegando antes da operação de leitura de um processo é colocada em uma fila (finita)

Para se comunicar com uma porta, um emissor necessita conhecer o Endereço IP e a Porta do processo receptor

- Combinação de endereço IP e porta é chamado de socket

NÚMERO DE PORTAS EM TRÊS GRUPOS

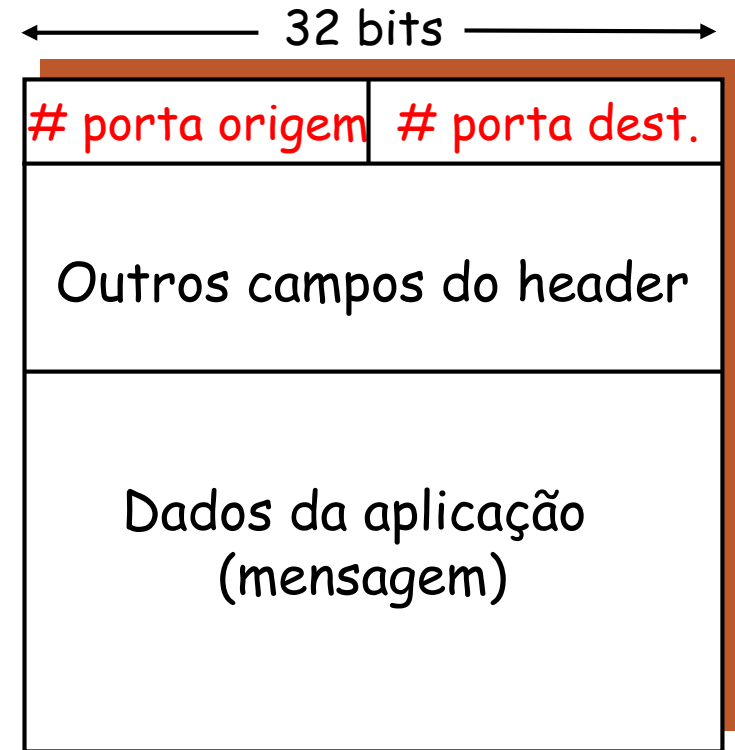
Faixa	Propósito
1 .. 1023	Portas bem conhecidas são atribuídas pela Internet Assigned Numbers Authority (IANA)
1024 .. 49151	Portas registradas
49152 .. 65535	Portas dinâmicas

- Servidores são normalmente conhecidos por portas bem conhecidas (por exemplo, 80 para HTTP)
- Portas dinâmicas podem ser usados por qualquer processos (normalmente usados por processos clientes)

MULTIPLEXAÇÃO/DEMULTIPLEXAÇÃO

Baseado no número da porta do emissor e receptor, e endereços IP

- Números de portas origem e destino em cada segmento (16 bits: 0..65535)
- lembrete: número de portas bem conhecidas para aplicações específicas (0..1024)



Formato do segmento TCP/UDP

DEMULTIPLEXAÇÃO NÃO ORIENTADA À CONEXÃO

Cria sockets com números de porta:

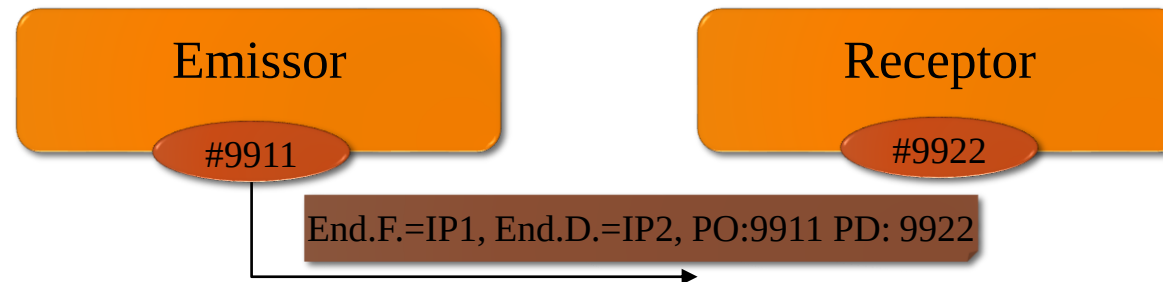
- Emissor: `mySocket1 = socket.socket(socket.AF_INET, SOCK_DGRAM)`
`mySocket1.bind((IP1, 9911))`
- Receptor: `mySocket2 = socket.socket(socket.AF_INET, SOCK_DGRAM)`
`mySocket2.bind((IP2, 9922))`

Socket UDP identificado pela porta que ele utiliza. No envio:

- `dest = (HOST, PORT)` # Destino da mensagem
- `mySocket1.sendto(message, dest)`

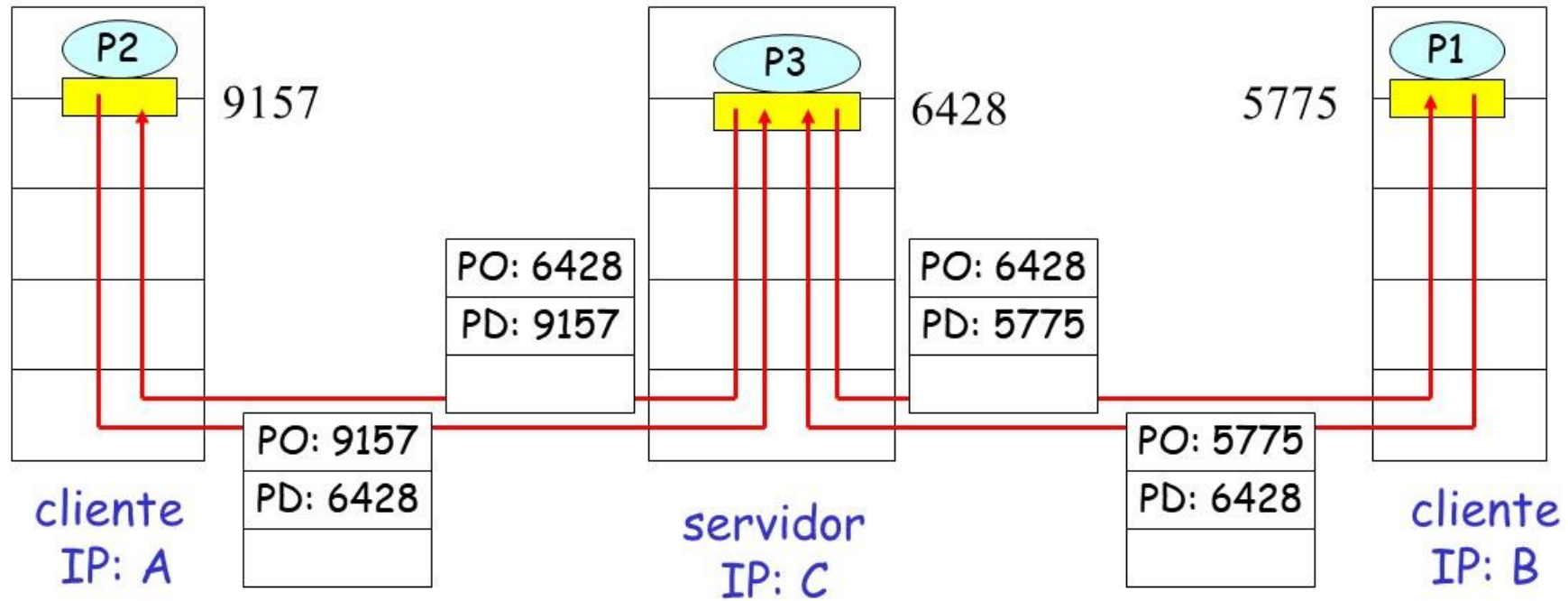
Na recepção o hospedeiro:

- Verifica o número da porta de destino no segmento
- Direciona o segmento UDP para o socket com este número de porta



DEMULTIPLEXAÇÃO NÃO ORIENTADA À CONEXÃO

```
mySocket3 = socket.socket(socket.AF_INET, SOCK_DGRAM)
mySocket3.bind((IPc, 6428))
```



PO: Porta de Origem
PD: Porta de Destino

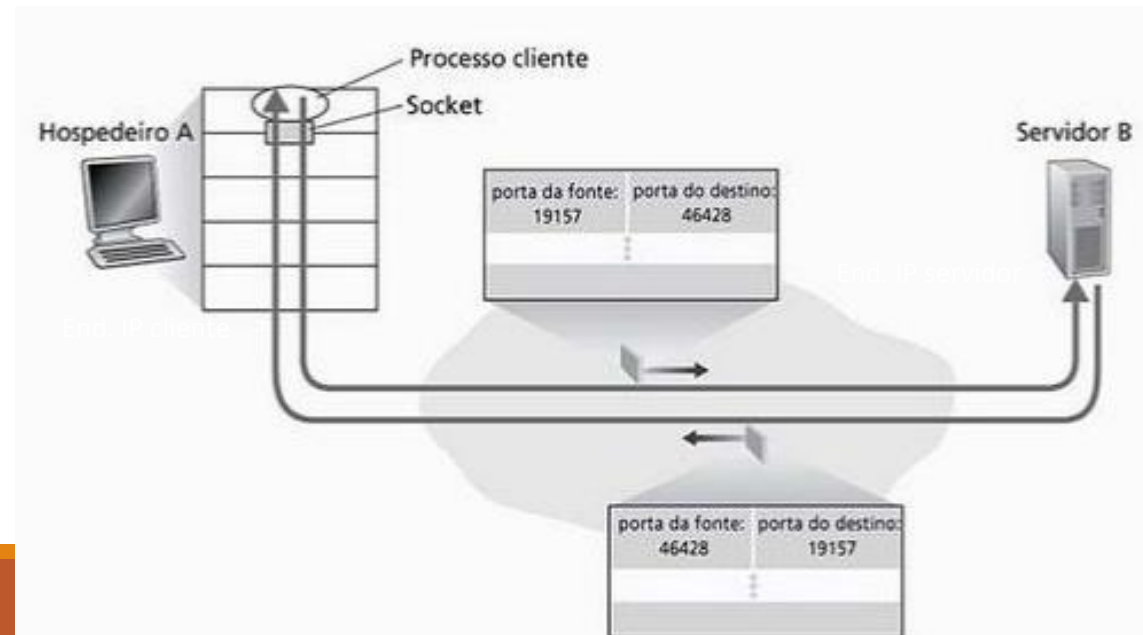
DEMUX ORIENTADA À CONEXÃO

No servidor (servidor.com.br) é instanciado um ServerSocket:

- `welcomeSocket=socket.socket(socket.AF_INET, socket.SOCK_STREAM)`
- `welcomeSocket.bind(("servidor.com.br", 46428))`
- `welcomeSocket.listen()`
- `conn, addr = s.accept()`

No cliente se comunicando com servidor:

- `clientSocket= socket.socket(socket.AF_INET, socket.SOCK_STREAM)`
- `clientSocket.connect((HOST, PORT))`

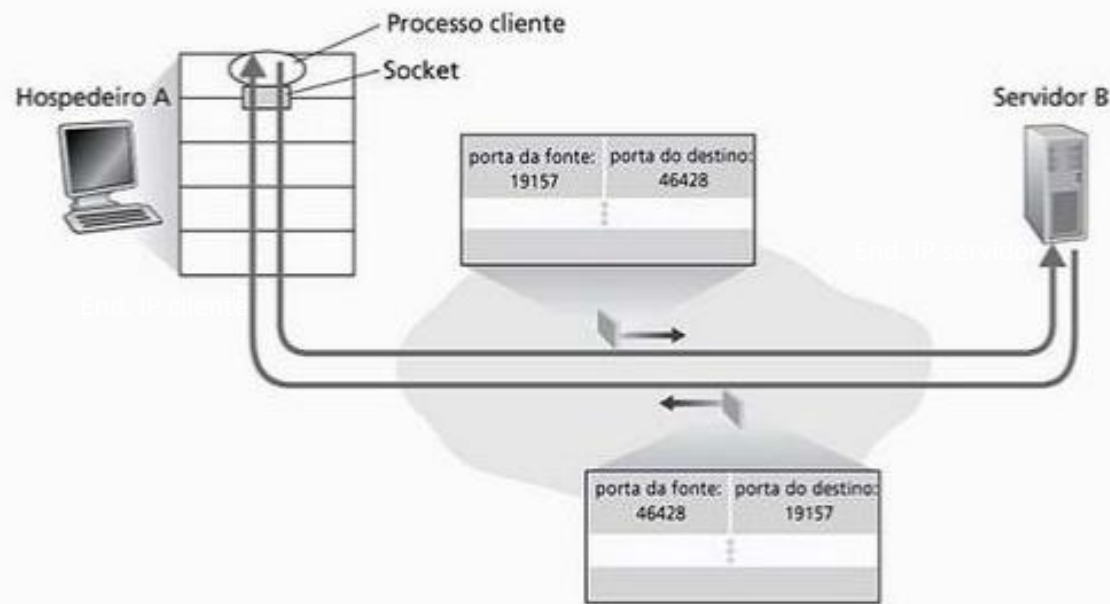


DEMUX ORIENTADA À CONEXÃO

Socket de conexão

- Instanciado no estabelecimento da conexão, é identificado por 4 valores: (End. IP de origem, End. porta de origem, Endereço IP de destino, End. porta de destino)
- Hospedeiro receptor usa os quatro valores para direcionar o segmento ao socket apropriado

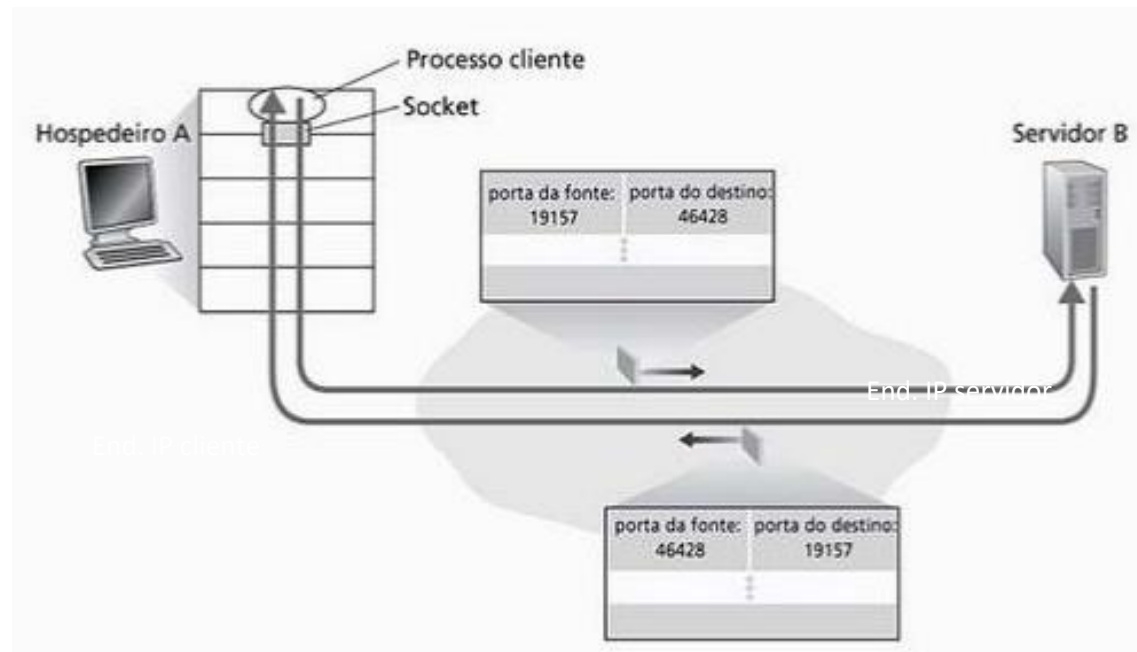
```
TCP 150.162.59.211:64347 162.125.34.129:https ESTABLISHED 3020
```



DEMUX ORIENTADA À CONEXÃO

Hospedeiro servidor pode suportar vários sockets TCP simultâneos:

- Cada socket de conexão é identificado pelos seus próprios 4 valores: Endereço Origem, Porta Origem, Endereço Destino, Porta Destino
- Servidores Web possuem sockets diferentes para cada cliente conectado



DEMUX ORIENTADA À CONEXÃO

